

烟幕干扰弹的投放策略研究

— 2025 年全国大学生数学建模竞赛 A 题

队伍编号: _____

队员 1: _____

队员 2: _____

队员 3: _____

摘要

本文研究了无人机投放烟幕干扰弹对来袭导弹实施遮蔽的最优策略问题。通过建立导弹运动模型、无人机运动模型、烟幕干扰弹抛体运动模型和烟幕云团运动模型，构建了完整的数学模型框架。

针对问题一，给定 FY1 无人机以 120 m/s 朝向假目标飞行，在 1.5 s 后投放 1 枚弹、3.6 s 后起爆的条件下，通过精确计算导弹轨迹、云团轨迹和遮蔽几何关系，得出有效遮蔽时长为 **X.XX 秒**。

针对问题二，以无人机飞行方向、飞行速度、投放时刻和起爆延迟为决策变量，建立了以有效遮蔽时长为目标的非线性优化模型。采用粒子群优化算法（PSO）求解，得到最优策略下遮蔽时长为 **X.XX 秒**，相比问题一提升了 **XX%**。

针对问题三，FY1 投放 3 枚弹，考虑投放间隔约束 (≥ 1 s)，建立了 8 维优化模型。通过 PSO 优化，实现了 3 枚弹的时间接力遮蔽，总有效遮蔽时长达 **X.XX 秒**。

针对问题四，FY1、FY2、FY3 各投放 1 枚弹，建立了 12 维协同优化模型。通过分阶段优化策略，充分利用 3 架无人机的空间分布优势，总遮蔽时长达 **X.XX 秒**。

针对问题五，5 架无人机各至多投放 3 枚弹干扰 3 枚导弹，建立了 40 维大规模优化模型。采用分层优化策略：首先基于几何关系进行任务分配，然后对每枚导弹独立优化投放策略。最终实现对 M1、M2、M3 的总遮蔽时长分别为 **X.XX 秒**、**X.XX 秒**、**X.XX 秒**。

本文还进行了灵敏度分析，考察了云团有效半径、有效遮蔽时间、云团下沉速度等参数对结果的影响，验证了模型的稳健性。

关键词：烟幕干扰弹；遮蔽策略；粒子群优化；抛体运动；视线遮挡

目录

1 问题重述

1.1 问题背景

烟幕干扰弹主要通过化学燃烧或爆炸分散形成烟幕或气溶胶云团，在目标前方特定空域形成遮蔽，干扰敌方导弹，具有成本低、效费比高等优点。随着烟幕干扰技术的不断发展，现已有多种投放方式完成烟幕干扰弹的定点精确抛撒，即在抛撒前能精确控制烟幕干扰弹到达预定位置，通过时间引信时序控制起爆时间。

本问题考虑运用无人机完成烟幕干扰弹的投放策略设计。具有长续航能力的无人机挂载某型烟幕干扰弹在特定空域巡飞，受领任务后，无人机投放烟幕干扰弹在来袭武器和保护目标之间形成烟幕遮蔽。

1.2 已知条件

1.2.1 作战环境

来袭武器为空地导弹，该型导弹飞行速度 300 m/s 。导弹的飞行方向直指一个为掩护某半径 7 m 、高 10 m 的圆柱形固定目标而专门设置的假目标。以假目标为原点，水平面为 xy 平面，真目标下底面的圆心为 $(0, 200, 0)$ 。

1.2.2 初始位置

警戒雷达发现来袭导弹时，3 枚导弹和 5 架无人机的位置信息如表 1 所示。

表 1 导弹与无人机初始位置

编号	x 坐标 (m)	y 坐标 (m)	z 坐标 (m)
M1	20000	0	2000
M2	19000	600	2100
M3	18000	-600	1900
FY1	17800	0	1800
FY2	12000	1400	1400
FY3	6000	-3000	700
FY4	11000	2000	1800
FY5	13000	-2000	1300

1.2.3 关键参数

表 2 关键参数汇总

参数	数值	说明
v_m	300 m/s	导弹飞行速度
v_u	70~140 m/s	无人机速度范围
v_c	3 m/s	烟幕云团下沉速度
R	10 m	云团有效遮蔽半径
T_{eff}	20 s	有效遮蔽时间窗口
Δt_{min}	1 s	同一无人机两枚弹最小投放间隔
g	9.8 m/s ²	重力加速度

1.3 五个子问题

问题 1: 利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰, 若 FY1 以 120 m/s 的速度朝向假目标方向飞行, 受领任务 1.5 s 后即投放 1 枚烟幕干扰弹, 间隔 3.6 s 后起爆。请给出烟幕干扰弹对 M1 的有效遮蔽时长。

问题 2: 利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰, 确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点, 使得遮蔽时间尽可能长。

问题 3: 利用无人机 FY1 投放 3 枚烟幕干扰弹, 实施对 M1 的干扰。请给出烟幕干扰弹的投放策略。

问题 4: 利用 FY1、FY2、FY3 等 3 架无人机, 各投放 1 枚烟幕干扰弹, 实施对 M1 的干扰。请给出烟幕干扰弹的投放策略。

问题 5: 利用 5 架无人机, 每架无人机至多投放 3 枚烟幕干扰弹, 实施对 M1、M2、M3 等 3 枚来袭导弹的干扰。请给出烟幕干扰弹的投放策略。

2 问题分析

2.1 核心物理过程分析

烟幕干扰弹的完整运动过程可分为三个阶段:

- (1) **随无人机飞行阶段** ($t = 0 \sim t_d$): 烟幕干扰弹挂载在无人机上, 随无人机以相同速度和方向飞行。
- (2) **抛体运动阶段** ($t = t_d \sim t_e$): 烟幕干扰弹脱离无人机后, 在重力作用下做抛体运动, 初速度等于无人机速度。

- (3) 云团下沉阶段 ($t = t_e \sim t_e + 20$): 起爆后形成球状烟幕云团, 以 3 m/s 匀速下沉, 20 s 内提供有效遮蔽。

2.2 遮蔽条件的几何本质

关键认识: 有效遮蔽不是指导弹到云团的距离小于 10 m , 而是烟幕云团遮挡了导弹对真目标的视线。

由于导弹飞行方向指向假目标 (原点), 而真目标位于 $(0, 200, 0)$, 烟幕干扰弹的目的是在导弹和真目标之间形成遮蔽屏障。因此, 有效遮蔽的几何条件是: 云团中心到导弹与真目标连线段的距离不超过云团有效遮蔽半径 $R = 10 \text{ m}$ 。

2.3 真目标的几何处理

真目标是半径 7 m 、高 10 m 的圆柱体。为精确建模, 需在圆柱体表面采样多个点, 检查云团是否遮挡了导弹到所有这些点的视线。本文采用在圆柱体侧面和上下底面均匀采样约 50 个点的方案。

3 模型假设

假设 1. 忽略空气阻力对烟幕干扰弹运动的影响。烟幕干扰弹在抛体运动阶段仅受重力作用。

假设 2. 重力加速度取 $g = 9.8 \text{ m/s}^2$, 方向垂直向下, 在整个运动过程中保持不变。

假设 3. 烟幕干扰弹脱离无人机后立即进入抛体运动, 初速度等于无人机速度, 无过渡阶段。

假设 4. 烟幕云团起爆后瞬时形成球状, 且在整个有效遮蔽时间内保持球形, 有效遮蔽半径恒为 10 m 。

假设 5. 烟幕浓度在有效遮蔽范围内均匀分布, 云团中心 10 m 范围外的烟幕浓度不足以提供遮蔽。

假设 6. 导弹飞行方向始终指向假目标 (原点), 飞行速度恒定为 300 m/s , 不考虑导弹的机动规避。

假设 7. 无人机受领任务后可瞬时调整飞行方向, 之后保持等高度匀速直线飞行, 航向和速度不再调整。

假设 8. 不考虑地球曲率、大气折射和风速对烟幕云团运动的影响。

4 符号说明

本文使用的主要符号及其含义如表 3 所示。

表 3 符号说明

符号	含义	单位
$\mathbf{P}_{m0}^{(i)}$	第 i 枚导弹的初始位置	m
$\mathbf{P}_{u0}^{(j)}$	第 j 架无人机的初始位置	m
v_m	导弹飞行速度 (300)	m/s
$v_u^{(j)}$	第 j 架无人机的飞行速度	m/s
$\mathbf{d}_u^{(j)}$	第 j 架无人机的飞行方向 (单位向量)	—
θ_j	第 j 架无人机的飞行方向角	rad
$t_d^{(k)}$	第 k 枚烟幕弹的投放时刻	s
$t_e^{(k)}$	第 k 枚烟幕弹的起爆时刻	s
$\Delta t^{(k)}$	第 k 枚弹从投放到起爆的时间间隔	s
$\mathbf{P}_d^{(k)}$	第 k 枚弹的投放点位置	m
$\mathbf{P}_e^{(k)}$	第 k 枚弹的起爆点位置	m
$\mathbf{P}_c^{(k)}(t)$	第 k 枚弹云团中心在时刻 t 的位置	m
g	重力加速度 (9.8)	m/s ²
v_c	云团下沉速度 (3)	m/s
R	云团有效遮蔽半径 (10)	m
T_{eff}	有效遮蔽时间窗口 (20)	s
$\mathbf{T}$	真目标中心点	m
r_T	真目标圆柱半径 (7)	m
h_T	真目标圆柱高度 (10)	m
T_{shield}	总有效遮蔽时长	s

5 模型建立

5.1 坐标系建立

以假目标位置为原点 O ，水平面为 xy 平面， z 轴垂直向上。在此坐标系下：

- 假目标： $O = (0, 0, 0)$
- 真目标下底面圆心： $C = (0, 200, 0)$
- 真目标中心点： $\mathbf{T} = (0, 200, 5)$

5.2 导弹运动模型

导弹 i 的初始位置为 $\mathbf{P}_{m0}^{(i)}$ ，飞行方向指向原点 O ，方向向量为：

$$\mathbf{d}_m^{(i)} = \frac{\mathbf{O} - \mathbf{P}_{m0}^{(i)}}{|\mathbf{O} - \mathbf{P}_{m0}^{(i)}|} = \frac{(-x_{m0}^{(i)}, -y_{m0}^{(i)}, -z_{m0}^{(i)})}{\sqrt{(x_{m0}^{(i)})^2 + (y_{m0}^{(i)})^2 + (z_{m0}^{(i)})^2}} \quad (1)$$

导弹 i 在时刻 t 的位置：

$$\mathbf{P}_m^{(i)}(t) = \mathbf{P}_{m0}^{(i)} + v_m \cdot t \cdot \mathbf{d}_m^{(i)} \quad (2)$$

各导弹的速度分量如表 4 所示。

表 4 各导弹速度分量

编号	$ \mathbf{P}_{m0} $ (m)	v_{mx} (m/s)	v_{my} (m/s)	v_{mz} (m/s)
M1	20099.75	-298.50	0	-29.85
M2	19121.98	-297.04	-9.41	-32.84
M3	18033.30	-299.45	9.98	-31.61

5.3 无人机运动模型

无人机 j 的初始位置为 $\mathbf{P}_{u0}^{(j)}$ ，飞行方向角为 θ_j （水平面内），飞行方向单位向量：

$$\mathbf{d}_u^{(j)} = (\cos \theta_j, \sin \theta_j, 0) \quad (3)$$

无人机 j 在时刻 t 的位置（ $t \geq 0$ ，以受领任务为 $t = 0$ 时刻）：

$$\mathbf{P}_u^{(j)}(t) = \mathbf{P}_{u0}^{(j)} + v_u^{(j)} \cdot t \cdot \mathbf{d}_u^{(j)} \quad (4)$$

约束条件： $v_u^{(j)} \in [70, 140]$ m/s，高度保持不变。

5.4 烟幕干扰弹运动模型

5.4.1 投放点计算

弹 k 由无人机 j 在时刻 $t_d^{(k)}$ 投放，投放点即为无人机在该时刻的位置：

$$\mathbf{P}_d^{(k)} = \mathbf{P}_u^{(j)}(t_d^{(k)}) = \mathbf{P}_{u0}^{(j)} + v_u^{(j)} \cdot t_d^{(k)} \cdot \mathbf{d}_u^{(j)} \quad (5)$$

5.4.2 抛体运动模型

弹 k 脱离无人机后，初速度等于无人机速度 $\mathbf{v}_0^{(k)} = v_u^{(j)} \cdot \mathbf{d}_u^{(j)}$ 。在重力作用下，弹 k 在时刻 t ($t_d^{(k)} \leq t \leq t_e^{(k)}$) 的位置：

$$\mathbf{P}_b^{(k)}(t) = \mathbf{P}_d^{(k)} + \mathbf{v}_0^{(k)} \cdot (t - t_d^{(k)}) + \frac{1}{2} \mathbf{g}(t - t_d^{(k)})^2 \quad (6)$$

其中 $\mathbf{g} = (0, 0, -g) = (0, 0, -9.8) \text{ m/s}^2$ 。

5.4.3 起爆点计算

弹 k 在时刻 $t_e^{(k)} = t_d^{(k)} + \Delta t^{(k)}$ 起爆，起爆点位置：

$$\mathbf{P}_e^{(k)} = \mathbf{P}_d^{(k)} + \mathbf{v}_0^{(k)} \cdot \Delta t^{(k)} + \frac{1}{2} \mathbf{g}(\Delta t^{(k)})^2 \quad (7)$$

将式 (5) 代入，展开得：

$$\mathbf{P}_e^{(k)} = \mathbf{P}_{u0}^{(j)} + v_u^{(j)} \cdot t_e^{(k)} \cdot \mathbf{d}_u^{(j)} + \left(0, 0, -\frac{1}{2}g(\Delta t^{(k)})^2\right) \quad (8)$$

5.4.4 云团运动模型

起爆后，烟幕云团以 $v_c = 3 \text{ m/s}$ 匀速下沉。云团 k 在时刻 t ($t_e^{(k)} \leq t \leq t_e^{(k)} + T_{\text{eff}}$) 的中心位置：

$$\mathbf{P}_c^{(k)}(t) = \mathbf{P}_e^{(k)} + (0, 0, -v_c) \cdot (t - t_e^{(k)}) \quad (9)$$

即：

$$\mathbf{P}_c^{(k)}(t) = (x_e^{(k)}, y_e^{(k)}, z_e^{(k)} - v_c(t - t_e^{(k)})) \quad (10)$$

5.5 有效遮蔽条件模型

5.5.1 遮蔽几何分析

有效遮蔽的本质是：烟幕云团位于导弹与真目标之间，遮挡了导弹对真目标的视线。如图 1 所示。

图 1 烟幕遮蔽几何示意图

设：

- 导弹位置: $\mathbf{M} = \mathbf{P}_m(t)$
- 云团中心: $\mathbf{C} = \mathbf{P}_c(t)$
- 真目标中心: $\mathbf{T} = (0, 200, 5)$

5.5.2 点到线段距离计算

导弹与真目标连线段的参数方程:

$$\mathbf{L}(s) = \mathbf{M} + s(\mathbf{T} - \mathbf{M}), \quad s \in [0, 1] \quad (11)$$

令 $\mathbf{v} = \mathbf{T} - \mathbf{M}$, $\mathbf{w} = \mathbf{C} - \mathbf{M}$, 则最近点参数:

$$s^* = \frac{\mathbf{w} \cdot \mathbf{v}}{\mathbf{v} \cdot \mathbf{v}} \quad (12)$$

约束到 $[0, 1]$ 区间:

$$s^* = \max(0, \min(1, s^*)) \quad (13)$$

云团中心到视线段的距离:

$$d(t) = |\mathbf{C} - (\mathbf{M} + s^* \cdot \mathbf{v})| = |\mathbf{w} - s^* \cdot \mathbf{v}| \quad (14)$$

5.5.3 有效遮蔽判据

$$\text{Shields}(t) = \begin{cases} 1, & \text{if } d(t) \leq R \text{ and } t_e \leq t \leq t_e + T_{\text{eff}} \\ 0, & \text{otherwise} \end{cases} \quad (15)$$

5.5.4 考虑圆柱体目标的精确模型

真目标是半径 $r_T = 7$ m、高 $h_T = 10$ m 的圆柱体。为精确建模, 在圆柱体表面均匀采样 N 个点 $\{\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_N\}$, 有效遮蔽要求对所有采样点的视线都被遮挡:

$$\text{Shields}(t) = \begin{cases} 1, & \text{if } \max_{k=1, \dots, N} d_k(t) \leq R \\ 0, & \text{otherwise} \end{cases} \quad (16)$$

其中 $d_k(t)$ 为云团中心到导弹与第 k 个采样点连线段的距离。

采样方案: 侧面取 $n_{\text{side}} = 16$ 个角度、3 个高度层, 共 48 个点; 上下底面各取 $n_{\text{top}} = 8$ 个角度、3 个径向位置, 共 48 个点。总计约 96 个采样点。

5.6 优化目标函数

5.6.1 单弹遮蔽时长

$$T_{\text{shield}}^{(k)} = \int_{t_e^{(k)}}^{t_e^{(k)} + T_{\text{eff}}} \mathbb{I}[d^{(k)}(t) \leq R] dt \quad (17)$$

5.6.2 多弹联合遮蔽时长

多枚弹的遮蔽时间可能重叠, 总有效遮蔽时长为各弹遮蔽时间区间的并集长度:

$$T_{\text{total}} = \text{measure} \left(\bigcup_{k=1}^N \{t : d^{(k)}(t) \leq R, t_e^{(k)} \leq t \leq t_e^{(k)} + T_{\text{eff}}\} \right) \quad (18)$$

其中 $\text{measure}(\cdot)$ 表示集合的 Lebesgue 测度 (即并集的总长度)。

5.6.3 多导弹联合遮蔽

对于问题 5, 需同时干扰 M1、M2、M3 三枚导弹。优化目标为对所有导弹的总遮蔽时长之和:

$$T_{\text{total}} = \sum_{i=1}^3 T_{\text{total}}^{(i)} \quad (19)$$

6 模型求解

6.1 问题 1 求解

6.1.1 已知条件

- FY1 初始位置: $\mathbf{P}_{u0} = (17800, 0, 1800)$
- 飞行速度: $v_u = 120 \text{ m/s}$
- 飞行方向: 朝向假目标 (原点), 水平分量方向 $\mathbf{d}_u = (-1, 0, 0)$
- 投放时刻: $t_d = 1.5 \text{ s}$

- 起爆延迟: $\Delta t = 3.6 \text{ s}$, 即 $t_e = 5.1 \text{ s}$

6.1.2 逐步计算

Step 1: 计算投放点

$$\mathbf{P}_d = (17800, 0, 1800) + 120 \times 1.5 \times (-1, 0, 0) = (17620, 0, 1800) \quad (20)$$

Step 2: 计算起爆点

弹的初速度 $\mathbf{v}_0 = (-120, 0, 0) \text{ m/s}$, 从投放到起爆的时间间隔 $\Delta t = 3.6 \text{ s}$ 。

$$\begin{aligned} \mathbf{P}_e &= (17620, 0, 1800) + (-120, 0, 0) \times 3.6 + \frac{1}{2}(0, 0, -9.8) \times 3.6^2 \\ &= (17620 - 432, 0, 1800 - 63.504) \\ &= (17188, 0, 1736.496) \end{aligned} \quad (21)$$

Step 3: 计算导弹 M1 轨迹

M1 初始位置 $(20000, 0, 2000)$, 方向向量 $\mathbf{d}_m \approx (-0.9950, 0, -0.0995)$ 。

$$\mathbf{P}_m(t) = (20000 - 298.50t, 0, 2000 - 29.85t) \quad (22)$$

Step 4: 计算云团轨迹

$$\mathbf{P}_c(t) = (17188, 0, 1736.496 - 3(t - 5.1)) = (17188, 0, 1751.796 - 3t) \quad (23)$$

Step 5: 计算遮蔽距离

真目标中心 $\mathbf{T} = (0, 200, 5)$ 。

视线方向: $\mathbf{v}(t) = \mathbf{T} - \mathbf{P}_m(t) = (-20000 + 298.50t, 200, -1995 + 29.85t)$

云团到导弹向量: $\mathbf{w}(t) = \mathbf{P}_c(t) - \mathbf{P}_m(t) = (-2812 + 298.50t, 0, -248.204 + 26.85t)$

代入式 (12) 和 (14) 计算 $d(t)$ 。

Step 6: 数值求解

在 $t \in [5.1, 25.1]$ 范围内, 以 $\Delta t = 0.01 \text{ s}$ 步长遍历, 统计 $d(t) \leq 10$ 的时间。

计算结果: 有效遮蔽时长为 **X.XX s**。

6.1.3 问题 1 结果可视化

(a) 3D 轨迹图

(b) 遮蔽距离-时间曲线

图 2 问题 1 计算结果

6.2 问题 2 求解

6.2.1 优化模型

以 FY1 的飞行方向角 θ 、飞行速度 v_u 、投放时刻 t_d 和起爆延迟 Δt 为决策变量，建立优化模型：

$$\max_{\theta, v_u, t_d, \Delta t} T_{\text{shield}}(\theta, v_u, t_d, \Delta t) \quad (24)$$

$$\text{s.t.} \quad \begin{cases} 0 \leq \theta < 2\pi \\ 70 \leq v_u \leq 140 \\ t_d \geq 0 \\ \Delta t \geq 0 \\ t_e + 20 \leq t_{\text{arrive}} \end{cases} \quad (25)$$

其中 t_{arrive} 为导弹到达原点的时间。

6.2.2 粒子群优化算法 (PSO)

算法原理：

粒子群优化算法模拟鸟群觅食行为，每个粒子代表一个候选解。粒子根据个体最优位置和全局最优位置更新速度和位置。

算法参数设置：

表 5 PSO 算法参数

参数	取值
粒子数	100
最大迭代次数	300
惯性权重 w	$0.9 \rightarrow 0.4$ (线性递减)
学习因子 c_1	2.0
学习因子 c_2	2.0

速度更新公式：

$$\mathbf{v}_i^{(k+1)} = w \cdot \mathbf{v}_i^{(k)} + c_1 r_1 (\mathbf{p}_i^{\text{best}} - \mathbf{x}_i^{(k)}) + c_2 r_2 (\mathbf{g}^{\text{best}} - \mathbf{x}_i^{(k)}) \quad (26)$$

其中 $r_1, r_2 \in [0, 1]$ 为随机数, $\mathbf{p}_i^{\text{best}}$ 为个体最优位置, $\mathbf{g}^{\text{best}}$ 为全局最优位置。

位置更新公式：

$$\mathbf{x}_i^{(k+1)} = \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)} \quad (27)$$

6.2.3 收敛性分析

图 3 问题 2 PSO 收敛曲线

6.2.4 问题 2 结果

通过 PSO 优化, 得到最优策略如表 6 所示。

表 6 问题 2 最优策略

参数	最优值
飞行方向角 θ	XX°
飞行速度 v_u	XX m/s
投放时刻 t_d	XX s
起爆延迟 Δt	XX s
有效遮蔽时长	XX s

6.3 问题 3 求解

6.3.1 优化模型

FY1 投放 3 枚弹, 决策变量包括共享的飞行参数和各弹的投放参数:

$$\max_{\theta, v_u, t_{d,1}, \Delta t_1, t_{d,2}, \Delta t_2, t_{d,3}, \Delta t_3} T_{\text{total}} \quad (28)$$

约束条件:

$$\text{s.t.} \quad \begin{cases} 70 \leq v_u \leq 140 \\ t_{d,i+1} \geq t_{d,i} + 1, \quad i = 1, 2 \\ \Delta t_i \geq 0, \quad i = 1, 2, 3 \end{cases} \quad (29)$$

共 8 个决策变量。

6.3.2 求解策略

采用 8 维 PSO 搜索，适应度函数需计算 3 枚弹的遮蔽时间并集长度。

时间区间并集计算算法：

```

1 def union_length(intervals):
2     """计算多个时间区间的并集长度"""
3     if not intervals:
4         return 0
5     # 按起始时间排序
6     intervals.sort(key=lambda x: x[0])
7     merged = [intervals[0]]
8     for start, end in intervals[1:]:
9         if start <= merged[-1][1]:
10             merged[-1] = (merged[-1][0], max(merged[-1][1], end))
11         else:
12             merged.append((start, end))
13     return sum(end - start for start, end in merged)

```

6.3.3 问题 3 结果

表 7 问题 3 投放策略

弹编号	投放时刻 (s)	起爆延迟 (s)	投放点坐标	起爆点坐标	遮蔽时长 (s)
1	XX	XX	(XX,XX,XX)	(XX,XX,XX)	XX
2	XX	XX	(XX,XX,XX)	(XX,XX,XX)	XX
3	XX	XX	(XX,XX,XX)	(XX,XX,XX)	XX
5r 总有效遮蔽时长	XX				

(a) 3 枚弹的 3D 轨迹

(b) 遮蔽时间甘特图

图 4 问题 3 计算结果

6.4 问题 4 求解

6.4.1 优化模型

FY1、FY2、FY3 各投放 1 枚弹，每架无人机独立决策。决策变量共 12 个：

$$\max_{\{\theta_i, v_i, t_{d,i}, \Delta t_i\}_{i=1}^3} T_{\text{total}} \quad (30)$$

约束条件：

$$\text{s.t. } 70 \leq v_i \leq 140, \quad t_{d,i} \geq 0, \quad \Delta t_i \geq 0, \quad i = 1, 2, 3 \quad (31)$$

6.4.2 求解策略

12 维 PSO 搜索。由于 3 架无人机位置不同，搜索空间较大。可采用以下策略提高效率：

1. 分阶段优化：先优化 FY1（离 M1 最近），再优化 FY2 和 FY3
2. 多次运行：PSO 运行 10 次，取最优结果
3. 自适应惯性权重：根据迭代进展动态调整

6.4.3 问题 4 结果

表 8 问题 4 投放策略

无人机	方向角 (°)	速度 (m/s)	投放时刻 (s)	起爆延迟 (s)	遮蔽时长 (s)
FY1	XX	XX	XX	XX	XX
FY2	XX	XX	XX	XX	XX
FY3	XX	XX	XX	XX	XX
5r 总有效遮蔽时长	XX				

6.5 问题 5 求解

6.5.1 问题规模分析

- 5 架无人机，每架至多 3 枚弹 $\Rightarrow$ 最多 15 枚弹
- 需干扰 M1、M2、M3 三枚导弹

- 决策变量：5 个方向角 + 5 个速度 + 15 个投放时刻 + 15 个起爆延迟 = 最多 40 个

6.5.2 分层优化策略

第一层：任务分配

根据无人机与导弹的几何关系，将无人机分配给最有利的导弹：

表 9 任务分配方案

导弹	初始位置	分配的无人机	理由
M1	(20000,0,2000)	FY1, FY2	FY1 离 M1 最近
M2	(19000,600,2100)	FY4	FY4 在 M2 侧
M3	(18000,-600,1900)	FY3, FY5	FY5 在 M3 侧

第二层：独立优化

对每枚导弹，独立优化分配给它的无人机的投放策略。每组子问题的维度较低（8-16 维），PSO 可以有效求解。

6.5.3 问题 5 结果

表 10 问题 5 投放策略汇总

无人机	弹编号	方向角	速度	投放时刻	起爆延迟	遮蔽时长
FY1	1	XX	XX	XX	XX	XX
FY1	2	XX	XX	XX	XX	XX
FY2	1	XX	XX	XX	XX	XX
⋮	⋮	⋮	⋮	⋮	⋮	⋮
6r 对 M1 总遮蔽时长	XX					
6r 对 M2 总遮蔽时长	XX					
6r 对 M3 总遮蔽时长	XX					
6r 总遮蔽时长	XX					

7 灵敏度分析

为检验模型的稳健性，对关键参数进行灵敏度分析。以问题 2 的最优策略为基础，分别考察各参数变化对遮蔽时长的影响。

7.1 分析参数与方法

表 11 灵敏度分析参数设置

参数	基准值	变化范围	步长
云团有效半径 R	10 m	5~15 m	1 m
有效遮蔽时间 T_{eff}	20 s	10~30 s	2 s
云团下沉速度 v_c	3 m/s	1~5 m/s	0.5 m/s
导弹速度 v_m	300 m/s	250~350 m/s	10 m/s

对每个参数，在其变化范围内等距取值，固定其他参数为基准值，重新计算最优遮蔽时长。

7.2 灵敏度分析结果

- (a) 有效半径 R 的灵敏度

(b) 有效遮蔽时间 T_{eff} 的灵敏度
- (c) 云团下沉速度 v_c 的灵敏度

(d) 导弹速度 v_m 的灵敏度

图 5 灵敏度分析结果

7.3 灵敏度分析结论

1. 云团有效半径 R : 遮蔽时长对 R 最敏感。 R 每增加 1 m，遮蔽时长约增加 XX%。
2. 有效遮蔽时间 T_{eff} : T_{eff} 增大直接增加可用遮蔽时间窗口，但受限于导弹飞行时间。
3. 云团下沉速度 v_c : v_c 增大会使云团更快偏离视线，遮蔽时长略有下降。
4. 导弹速度 v_m : v_m 增大使导弹更快接近目标，可用遮蔽时间窗口缩短。

8 模型评价与改进

8.1 模型优点

1. 物理建模准确: 完整考虑了烟幕干扰弹的三阶段运动过程（随无人机飞行、抛体运动、云团下沉），模型方程严格推导。

2. **遮蔽条件合理**: 基于视线遮挡的几何模型, 而非简单的距离判据, 更符合烟幕干扰的物理本质。
3. **目标处理精细**: 考虑了真目标为圆柱体的几何特征, 通过表面多点采样提高遮蔽判断精度。
4. **算法选择恰当**: PSO 适合连续优化问题, 具有全局搜索能力; 分层策略有效降低了问题 5 的维度。
5. **可扩展性好**: 模型框架可方便地扩展到更多无人机和导弹的场景。

8.2 模型缺点

1. **忽略空气阻力**: 实际弹道会有偏差, 特别是高速运动时。
2. **未考虑风速影响**: 烟幕云团可能被风吹散或偏移。
3. **计算量较大**: 圆柱体多点采样和时间步进法增加了计算复杂度。
4. **PSO 可能陷入局部最优**: 高维问题中, 单次 PSO 运行可能无法找到全局最优。

8.3 改进方向

1. 引入空气阻力修正抛体运动模型, 使用阻力系数 C_d 修正弹道。
2. 考虑风速对云团运动的影响, 在云团运动方程中加入风速项。
3. 采用自适应 PSO 或混合算法 (如 PSO-GA 混合) 提高搜索效率。
4. 增加蒙特卡洛模拟, 评估参数不确定性对策略鲁棒性的影响。
5. 考虑烟幕云团的扩散模型, 使有效遮蔽半径随时间变化。

参考文献

- [1] 司守奎, 孙兆亮. 数学建模算法与应用 [M]. 北京: 国防工业出版社, 2015.
- [2] 姜启源, 谢金星, 叶俊. 数学模型 (第五版)[M]. 北京: 高等教育出版社, 2018.
- [3] Kennedy J, Eberhart R. Particle swarm optimization[C]// Proceedings of ICNN'95 - International Conference on Neural Networks. IEEE, 1995: 1942-1948.
- [4] Shi Y, Eberhart R. A modified particle swarm optimizer[C]// 1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE, 1995: 69-73.
- [5] Goldberg D E. Genetic Algorithms in Search, Optimization, and Machine Learning[M]. Reading, MA: Addison-Wesley, 1989.
- [6] 刘卫国. MATLAB 程序设计与应用 (第三版)[M]. 北京: 高等教育出版社, 2017.
- [7] 全国大学生数学建模竞赛组委会. 2025 年全国大学生数学建模竞赛 A 题 [Z]. 2025.

A 附录

A.1 附录 A：核心 Python 代码

Listing 1 基础物理计算函数

```

1 import numpy as np
2
3 # ===== 常量定义 =====
4 V_M = 300.0      # 导弹速度 (m/s)
5 V_C = 3.0        # 云团下沉速度 (m/s)
6 G = 9.8          # 重力加速度 (m/s^2)
7 R = 10.0         # 有效遮蔽半径 (m)
8 T_EFF = 20.0     # 有效遮蔽时间 (s)
9
10 # 初始位置
11 M1 = np.array([20000, 0, 2000], dtype=float)
12 M2 = np.array([19000, 600, 2100], dtype=float)
13 M3 = np.array([18000, -600, 1900], dtype=float)
14 FY1 = np.array([17800, 0, 1800], dtype=float)
15 FY2 = np.array([12000, 1400, 1400], dtype=float)
16 FY3 = np.array([6000, -3000, 700], dtype=float)
17 FY4 = np.array([11000, 2000, 1800], dtype=float)
18 FY5 = np.array([13000, -2000, 1300], dtype=float)
19 TARGET = np.array([0, 200, 5], dtype=float) # 真目标中心
20
21 def missile_pos(P_m0, t):
22     """计算导弹在时刻t的位置"""
23     d_m = -P_m0 / np.linalg.norm(P_m0)
24     return P_m0 + V_M * t * d_m
25
26 def uav_pos(P_u0, v_u, theta, t):
27     """计算无人机在时刻t的位置"""
28     d_u = np.array([np.cos(theta), np.sin(theta), 0])
29     return P_u0 + v_u * t * d_u

```

```

30
31 def bomb_detonate_point(P_u0, v_u, theta, t_d, delta_t):
32     """计算起爆点位置"""
33     d_u = np.array([np.cos(theta), np.sin(theta), 0])
34     P_d = P_u0 + v_u * t_d * d_u
35     v0 = v_u * d_u
36     g_vec = np.array([0, 0, -G])
37     return P_d + v0 * delta_t + 0.5 * g_vec * delta_t**2
38
39 def cloud_center(P_e, t, t_e):
40     """计算云团中心在时刻t的位置"""
41     return P_e + np.array([0, 0, -V_C]) * (t - t_e)
42
43 def shielding_distance(P_m, P_c, T_target):
44     """计算云团中心到导弹-目标视线段的距离"""
45     v = T_target - P_m
46     w = P_c - P_m
47     vv = np.dot(v, v)
48     if vv < 1e-10:
49         return np.linalg.norm(P_c - P_m)
50     s = np.dot(w, v) / vv
51     s = max(0, min(1, s))
52     Q = P_m + s * v
53     return np.linalg.norm(P_c - Q)

```

Listing 2 遮蔽时长计算函数

```

1 def compute_shield_duration(P_u0, v_u, theta, t_d, delta_t,
2                             P_m0, dt=0.01):
3     """计算单枚弹的有效遮蔽时长"""
4     t_e = t_d + delta_t
5     P_e = bomb_detonate_point(P_u0, v_u, theta, t_d, delta_t)
6
7     total = 0.0
8     t = t_e

```

```

9     while t <= t_e + T_EFF:
10         P_m = missile_pos(P_m0, t)
11         P_c = cloud_center(P_e, t, t_e)
12         d = shielding_distance(P_m, P_c, TARGET)
13         if d <= R:
14             total += dt
15         t += dt
16     return total
17
18 def compute_multi_shield(intervals, dt=0.01):
19     """计算多枚弹的总遮蔽时长（并集）"""
20     if not intervals:
21         return 0.0
22     intervals.sort(key=lambda x: x[0])
23     merged = [intervals[0]]
24     for start, end in intervals[1:]:
25         if start <= merged[-1][1]:
26             merged[-1] = (merged[-1][0], max(merged[-1][1], end))
27         else:
28             merged.append((start, end))
29     return sum(end - start for start, end in merged)

```

Listing 3 PSO 优化器

```

1 class PSO:
2     """粒子群优化器"""
3     def __init__(self, n_particles, n_dims, lb, ub, fitness_func):
4         self.n = n_particles
5         self.d = n_dims
6         self.lb = np.array(lb)
7         self.ub = np.array(ub)
8         self.fitness = fitness_func
9
10        self.positions = np.random.uniform(lb, ub, (n_particles,
            n_dims))

```

```
11     self.velocities = np.zeros((n_particles, n_dims))
12     self.pbest_pos = self.positions.copy()
13     self.pbest_val = np.full(n_particles, -np.inf)
14     self.gbest_pos = None
15     self.gbest_val = -np.inf
16
17     self.w_max, self.w_min = 0.9, 0.4
18     self.c1, self.c2 = 2.0, 2.0
19
20     def optimize(self, max_iter):
21         convergence = []
22         for it in range(max_iter):
23             w = self.w_max - (self.w_max - self.w_min) * it /
24                 max_iter
25
26             for i in range(self.n):
27                 fit = self.fitness(self.positions[i])
28                 if fit > self.pbest_val[i]:
29                     self.pbest_val[i] = fit
30                     self.pbest_pos[i] = self.positions[i].copy()
31                 if fit > self.gbest_val:
32                     self.gbest_val = fit
33                     self.gbest_pos = self.positions[i].copy()
34
35             convergence.append(self.gbest_val)
36
37             r1 = np.random.random((self.n, self.d))
38             r2 = np.random.random((self.n, self.d))
39             self.velocities = (w * self.velocities
40                               + self.c1 * r1 * (self.pbest_pos - self.
41                                   positions)
42                               + self.c2 * r2 * (self.gbest_pos - self.
43                                   positions))
44             self.positions += self.velocities
```



```

42         self.positions = np.clip(self.positions, self.lb, self.ub
43                                   )
44
45         if it % 50 == 0:
46             print(f"Iter {it}: Best = {self.gbest_val:.4f}")
47
48         return self.gbest_pos, self.gbest_val, convergence

```

Listing 4 圆柱体目标采样

```

1 def sample_cylinder_surface(C_center, r, h, n_side=16, n_top=8):
2     """在圆柱体表面采样点"""
3     points = []
4     # 侧面
5     for i in range(n_side):
6         angle = 2 * np.pi * i / n_side
7         for j in range(3):
8             z = h * j / 2
9             points.append([
10                 C_center[0] + r * np.cos(angle),
11                 C_center[1] + r * np.sin(angle),
12                 C_center[2] + z
13             ])
14     # 上底面
15     for i in range(n_top):
16         angle = 2 * np.pi * i / n_top
17         for rho in [0, r/2, r]:
18             points.append([
19                 C_center[0] + rho * np.cos(angle),
20                 C_center[1] + rho * np.sin(angle),
21                 C_center[2] + h
22             ])
23     # 下底面
24     for i in range(n_top):
25         angle = 2 * np.pi * i / n_top

```

```

26     for rho in [0, r/2, r]:
27         points.append([
28             C_center[0] + rho * np.cos(angle),
29             C_center[1] + rho * np.sin(angle),
30             C_center[2]
31         ])
32     return np.array(points)

```

Listing 5 可视化代码

```

1 import matplotlib.pyplot as plt
2 from mpl_toolkits.mplot3d import Axes3D
3
4 def plot_3d_trajectory(P_m0, v_u, theta, t_d, delta_t, P_u0, title="")
5     ):
6         """绘制3D轨迹图"""
7         fig = plt.figure(figsize=(10, 8))
8         ax = fig.add_subplot(111, projection='3d')
9
10        # 导弹轨迹
11        t_arrive = np.linalg.norm(P_m0) / V_M
12        t_missile = np.linspace(0, t_arrive, 1000)
13        traj_m = np.array([missile_pos(P_m0, t) for t in t_missile])
14        ax.plot(traj_m[:,0], traj_m[:,1], traj_m[:,2], 'r-', label='
15            Missile')
16
17        # 无人机轨迹
18        t_uav = np.linspace(0, t_d, 100)
19        traj_u = np.array([uav_pos(P_u0, v_u, theta, t) for t in t_uav])
20        ax.plot(traj_u[:,0], traj_u[:,1], traj_u[:,2], 'b-', label='UAV')
21
22        # 云团轨迹
23        P_e = bomb_detonate_point(P_u0, v_u, theta, t_d, delta_t)
24        t_e = t_d + delta_t
25        t_cloud = np.linspace(t_e, t_e + T_EFF, 200)

```

```
24     traj_c = np.array([cloud_center(P_e, t, t_e) for t in t_cloud])
25     ax.plot(traj_c[:,0], traj_c[:,1], traj_c[:,2], 'g-', label='Cloud
    ')
26
27     # 目标
28     ax.scatter(*TARGET, c='gold', s=100, marker='*', label='Target')
29     ax.scatter(0, 0, 0, c='black', s=100, marker='x', label='Decoy')
30
31     ax.set_xlabel('X (m)')
32     ax.set_ylabel('Y (m)')
33     ax.set_zlabel('Z (m)')
34     ax.set_title(title)
35     ax.legend()
36     plt.tight_layout()
37     plt.savefig('figures/trajectory.pdf', dpi=300)
38     plt.show()
```